

Este material é composto por trechos de dois livros diferentes que abordam a evolução e os fundamentos dos bancos de dados, com foco especial no surgimento e nas tecnologias NoSQL.

+1

Aqui estão os pontos principais resumidos:

1. A Transição do Relacional (SQL) para o NoSQL

Domínio do Modelo Relacional: Criado por Edgar F. "Ted" Codd em 1970, utiliza a linguagem SQL e as propriedades ACID para garantir transações seguras e íntegras.

+1

Limitações do SQL: Com a ascensão da internet na década de 90, o modelo relacional enfrentou dificuldades para escalar horizontalmente e lidar com dados não estruturados.

+2

Surgimento do NoSQL: O termo ganhou força em 2009 para descrever bancos que oferecem esquemas flexíveis e alta escalabilidade para Big Data.

+2

2. Propriedades de Transação: ACID vs. BASE

ACID (Tradicional): Foca em Atomicidade (tudo ou nada), Consistência (dados válidos), Isolamento (transações independentes) e Durabilidade (dados salvos permanentemente).

BASE (NoSQL): Muitos bancos NoSQL abrem mão de garantias rígidas de consistência em favor da disponibilidade. O conceito mais importante aqui é a Consistência Eventual, onde o sistema garante que, se não houver novas atualizações, todos os nós eventualmente terão o mesmo valor.

+2

3. O Teorema CAP

Fundamental para sistemas distribuídos, o teorema afirma que só é possível garantir duas destas três propriedades simultaneamente:

+1

Consistência (C): Todos os leitores veem a atualização imediatamente.

Disponibilidade (A): O sistema responde mesmo com falhas em alguns nós.

Tolerância a Partição (P): O sistema continua operando mesmo se a comunicação entre os nós for cortada.

4. Estratégias de Escalabilidade e Distribuição

Sharding: Técnica de dividir os dados entre vários servidores (nós) de um cluster para aumentar a capacidade de armazenamento e resposta.

Fragmentação: Pode ser Horizontal (dividir por linhas) ou Vertical (dividir por colunas) para otimizar o acesso local aos dados.

Replicação: Armazenar cópias dos mesmos dados em diferentes sites para garantir que o sistema continue funcionando se um servidor falhar.

+1

5. Tipos e Exemplos de Bancos NoSQL

O livro classifica os bancos conforme seu modelo de dados:

+1

Documentos: MongoDB (usa BSON/JSON) e CouchDB.

+1

Chave-Valor: Redis (em memória) e Amazon Dynamo.

+1

Colunar: Google BigTable, Cassandra e HBase.

+2

Grafos: Neo4j (focado em relacionamentos complexos).

6. Técnicas de Consulta em Sistemas Distribuídos

MapReduce: Modelo criado pelo Google para processar volumes massivos de dados em paralelo.

+1

Bloomjoin e Semijunção: Técnicas avançadas para reduzir o volume de dados enviados pela rede durante a junção de tabelas/coleções que estão em servidores diferentes.

+1

Esse resumo cobre a base teórica necessária para entender por que o NoSQL se tornou essencial para aplicações modernas de alta escala.

Detalhes Técnicos sobre Distribuição de Dados

Independência de Dados Distribuídos: O livro explica que o SGBD deve permitir que você escreva consultas sem saber como os dados foram fragmentados ou replicados. O sistema usa "nomes globais" compostos pelo nome local e o "site de nascimento" para localizar os dados.

+2

Fragmentação Horizontal vs. Vertical:

Horizontal: O sistema divide a tabela por linhas (tuplas), geralmente usando uma condição de seleção (ex: funcionários por cidade).

+1

Vertical: Divide a tabela por colunas, usando projeção. Para não perder dados, o sistema anexa um ID único (TID) em cada fragmento para conseguir juntá-los depois.

+2

O Problema do Sharding e Hashing: O texto detalha que a forma mais comum de distribuir dados é via hashing da chave primária. Porém, se um servidor (nó) cair, o cálculo de hash mod n quebra. A solução apresentada é o Hash Consistente, onde os nós e dados são organizados em um "anel", permitindo que novos servidores entrem ou saiam sem bagunçar todo o sistema.

+3

Conceitos de Performance (Junções Distribuídas)

Se o seu professor quiser apertar na prova, ele pode perguntar sobre como o banco economiza rede:

Semijunção: Em vez de enviar uma tabela inteira de um servidor para outro para fazer um JOIN, o banco envia apenas a lista de IDs. O outro servidor filtra o que é necessário e devolve só o resultado.

+1

Bloomjoin: É uma evolução da semijunção que usa um vetor de bits (mais leve que uma lista de IDs) para filtrar os dados antes de enviá-los pela rede, economizando muito processamento e banda.

Sincronização e Relógios

Consistência Eventual: O livro admite que o cliente pode ler um dado antigo por um tempo, mas garante que o sistema convergirá no futuro.

Vector Clocks: Como os servidores NoSQL são distribuídos, eles não podem confiar em um único relógio central. Eles usam "Relógios de Vetores" para descobrir qual versão de um dado é a mais recente em caso de conflitos.